

Manual Técnico

Arquitectura General

El sistema sigue el flujo siguiente

Código .glt

>>>> es el que entra y luego comienza por el recorrido de fases ósea :

FASE 1: LÉXICO JFlex Lexer.java

Lexer.jflex Produce: lista de Token errores léxicos
stream de Symbols

FASE 2: SINTÁCTICO CUP Parser.java + sym.java

Parser.cup Produce: NodoPrograma (raíz del AST)
NodoPrograma

FASE 3: INTÉRPRETE Java puro

ast.*.execute(Entorno) Produce: salida de consola

Entorno (tabla de símbolos) + errores semánticos

GUI (Swing)

Consola · Reportes HTML

>>> El flujo descriptivo es de esta manera por su no se entiende a totalidad las fases son las que hacen el clímax del programa

Patrones de Diseño Aplicados

Singleton

>>> las instancias de los 3 son los métodos que hacen leer cada uno de nuestros operadores tokens y errores

- `Compiler.getInstance()` — controla el estado de la compilación
- `ErrorManager.getInstance()` — acumula errores de todas las fases
- `TablaTokens.getInstance()` — acumula tokens del lexer

Template Method

>>> siguiendo las buenas prácticas se creó una clase padre nodo para poder abstraer por ejemplo línea y columna que se leerá para hacer un análisis

Nodo (abstract)

- └─ execute(Entorno): Object ← método plantilla
- └─ toAST(int nivel): String ← método plantilla

Cada subclase define su comportamiento concreto. `NodoExpresion` agrega `getValue(Entorno)` y su `execute()` delega en `getValue()`.

(Entorno encadenado)

>>> los nodos concatenan su lógica entre sí

Entorno global

- └─ Entorno función
 - └─ Entorno bloque if
 - └─ Entorno bloque for

`obtener(nombre)` sube por la cadena hasta encontrar la variable o retornar
`NO\_ENCONTRADO`.

Reportable

ErrorManager TablaTokens TablaSimbolos ReporteAST todos implementan
la interfaz `Reportable`, lo que permite a MainFramw mostrarlos de forma
uniforme.

Flujo de control basado en excepciones

BreakException ContinueException ReturnExceptio se propagan por la
pila de llamadas Java hasta el nodo correspondiente que las captura:

NodoFor captura BreakException y ContinueException

NodoFuncion captura ReturnException

Compiler captura las que escapan al nivel global (error semántico)

Jerarquía de Clases AST

Nodo (abstract, paquete ast)

```
| + int linea, columna  
| + Object execute(Entorno) abstract  
| + String toAST(int) abstract  
|
```

└─ NodoExpresion (abstract)

- | | + Object getValue(Entorno) abstract
- | | $\vdash \text{execute()} \rightarrow \text{getValue}()$
- | |
- | $\vdash \text{NodoLiteral}$ valor constante (int, float64, string, bool, rune, nil)
- | $\vdash \text{NodoIdentificador}$ lectura de variable
- | $\vdash \text{NodoBinario}$ + - * / % == != < > <= >= && ||
- | $\vdash \text{NodoUnario}$ - !
- | $\vdash \text{NodoLlamadaFuncion}$ invocación de función user-defined
- | $\vdash \text{NodoFmtPrintf}$ fmt.Println(args...)
- | $\vdash \text{NodoStrconvAtoi}$ strconv.Atoi(expr)
- | $\vdash \text{NodoStrconvParseFloat}$ strconv.ParseFloat(expr)
- | $\vdash \text{NodoReflectTypeOf}$ reflect.TypeOf(expr)
- |
- | $\vdash \text{NodoSentencia}$ (abstract)
- | | $\vdash \text{NodoBloque}$ { sentencias... } $\rightarrow$ nuevo Entorno hijo
- | | $\vdash \text{NodoDeclaracionVar}$ var id tipo = expr | id := expr
- | | $\vdash \text{NodoAsignacion}$ id = expr
- | | $\vdash \text{NodoAsignacionCompuesta}$ id += expr | id -= expr
- | | $\vdash \text{NodoIncrementoDecremento}$ id++ | id--
- | | $\vdash \text{NodoIf}$ if / else if / else
- | | $\vdash \text{NodoFor}$ for clásico | for condición
- | | $\vdash \text{NodoBreak}$ $\rightarrow$ lanza BreakException
- | | $\vdash \text{NodoContinue}$ $\rightarrow$ lanza ContinueException

- | └─ NodoExpresionSentencia envuelve expresión como sentencia
- |
- |─┐─ NodoFuncion declaración func
- |─┐─ NodoParametro parámetro de función
- |─┐─ NodoTipo tipo de dato
- |─┐─ NodoPrograma raíz: registra funciones + llama main()

Entorno (Tabla de Símbolos)

Estructura interna

java

```
class Entorno {
    Map<String, Simbolo>   tabla;     // variables locales
    Entorno            padre;     // ámbito superior
    String            nombreAmbito; // "Global" | "main" | ...
    // Solo en el global:
    Map<String, NodoFuncion>  funciones;
    StringBuilder       salidaConsola;
    List<Simbolo>        historicoSimbolos;
}
```

Métodos clave

Método.	Descripción
-----	-----

`declarar(nombre, tipo, valor)`	Declara en el ámbito ACTUAL
`obtener(nombre)`	Busca subiendo por padres
`obtenerTipo(nombre)`	Retorna el tipo declarado
`asignar(nombre, valor)`	Asigna donde fue declarada
`existeEnActual(nombre)`	Solo en el ámbito actual
`declararFuncion(nombre, nodo)`	En el global
`buscarFuncion(nombre)`	En el global
`agregarSalida(texto)`	Acumula fmt.Println
`getSalidaConsola()`	Toda la salida acumulada

Sistema de Errores

Categorías

Tipo.	Cuándo se genera.	Clase	
-----	-----	-----	
`LEXICO`	Carácter no reconocido por JFlex		
`ErrorManager.agregarLexico()`			
`SINTACTICO`	Estructura gramatical inválida		
`Parser.report_error()`			
`SEMANTICO`	Tipos incompatibles, var no declarada, etc.	`ErrorSemanticoException`	

Flujo de recuperación

1. El lexer registra el error léxico y ****continúa**** el análisis
1. CUP registra el error sintáctico y ****continúa**** hasta donde sea posible
1. Si hay errores sintácticos, el intérprete ****no ejecuta****
1. Los errores semánticos se registran en `ErrorManager` y la ejecución continúa cuando el error es recuperable (ej: variable no declarada → salta la sentencia)

Flujo de Compilación en Compiler.java

compilar(codigoFuente)

```

|
|  └─ reset() de ErrorManager, TablaTokens, Entorno
|
|  └─ new Lexer(StringReader)
|
|  └─ new Parser(lexer)
|    └─ parser.parse()
|      └─ CUP consume tokens del lexer (llama next_token())
|        └─ Construye el AST
|          └─ Retorna Symbol con NodoPrograma como .value
|
|  └─ tokens = lexer.getListTokens()
|
|  └─ Si errores sintácticos → STOP, devolver consola
|
|  └─ ast.execute(entornoGlobal)
|    └─ NodoPrograma registra funciones

```

```

|   |-- NodoPrograma llama main.execute()
|   |-- Cada nodo ejecuta su lógica, acumula salida en entorno
|
|-- consola.append(entornoGlobal.getSalidaConsola())
|
|-- return consola

```

Interfaz echa (Swing)

Componentes

Clase	Responsabilidad
`MainFrame`	Ventana principal, menús, toolbar, splitpane
`EditorTab`	Panel de edición con números de línea
`ConsolaPanel`	Área de salida inferior
`ReporteFrame`	Ventana HTML para reportes
`TabHeader`	Cabecera de pestaña con botón ×

Generación de Reportes HTML

Todos los reportes son cadenas HTML generadas por:

- `ErrorManager.generarReporteHTML()`
- `TablaTokens.generarReporteHTML()`
- `TablaSimbolos.generarReporteHTML()`
- `ReporteAST.generarReporteHTML()`

Extending para Fase 2 lo que me falta agregar en la siguiente parte

Para agregar soporte de switch/case slices structs y return:

Lexer.jflex ya tiene los tokens SWITCH CASE DEFAULT RETURN etc.

Parser.cup agregar producciones en sentencia declaracion_global

Ast crear NodoSwitch NodoReturn NodoSlice NodoStruct etc.

entorno/Entorno.java agregar soporte de slices con ArrayList<Object>

gui/MainFrame.java los reportes de Símbolos y AST ya están activos

Los paquetes excepciones ya tienen ReturnException lista para usar.

